

Architectural Paradigms of Artificial Intelligence in Macroscopic Grand Strategy Games

The engineering of Artificial Intelligence (AI) for macroscopic, map-based Grand Strategy Games (GSGs) represents a distinct and exceptionally rigorous discipline within the broader field of computational game theory and interactive software architecture. Unlike traditional Real-Time Strategy (RTS) titles, which fundamentally prioritize micro-tactical unit maneuvering within strictly bounded Euclidean arenas, macroscopic GSGs are designed to simulate the sprawling geopolitical, economic, and military trajectories of entire human or alien civilizations across spans of centuries.¹ The computational entities operating within these environments—whether representing nations in the mid-20th century, medieval dynasties, or interstellar empires—do not merely solve isolated combat scenarios. Instead, they must concurrently evaluate cascading relational databases, balance long-term macroeconomic deficits, negotiate deeply abstract non-Euclidean spatial topologies, and manage tens of thousands of autonomous agents, all without violating the temporal serialization of a deterministic state machine.¹

The underlying complexity of this task is compounded by the strict performance budgets dictated by consumer hardware limitations. In an environment where the state space dwarfs even traditional benchmarks like Chess or Go, the AI cannot rely on exhaustive search algorithms like Monte Carlo Tree Search (MCTS) for every decision, as the branching factor of a grand strategy simulation is effectively infinite.¹ Furthermore, because these games typically feature robust multiplayer ecosystems, the underlying architecture must synchronize the simulation perfectly across all clients. This necessitates a rigid adherence to deterministic lockstep networking models, which inherently restrict how and when asynchronous multithreading can be deployed to parallelize cognitive workloads.⁶

To achieve a believable, computationally efficient, and deterministic macroscopic intelligence, modern proprietary engines—such as the Clausewitz and Jomini engines utilized by Paradox Interactive—have pioneered highly specialized architectural paradigms. These systems decouple the cognitive evaluation phase from the state-mutation phase to prevent concurrent race conditions through read-lock and write-lock matrices.⁸ Furthermore, these architectures artificially constrain their own algorithmic omniscience, utilizing data-driven personality matrices, stochastic utility scoring pipelines, and modulo-based temporal scheduling to simulate human fallibility and ensure the processing loop never stalls.⁹ The following comprehensive analysis provides an exhaustive technical dissection of the algorithmic structures, concurrency models, pathfinding topologies, and mathematical frameworks that drive macroscopic AI in the most advanced grand strategy architectures to date.

Macroscopic AI Decision Frameworks and Utility Scoring Pipelines

The foundational requirement for any AI operating within a grand strategy environment is the capacity to navigate a procedurally vast decision space while maintaining a distinct, recognizable behavioral profile. In traditional game development, AI architectures frequently rely on rigid Finite State Machines (FSMs), Hierarchical Behavior Trees (HBTs), or Goal-Oriented Action Planning (GOAP). While effective in constrained environments, these methodologies prove fundamentally brittle and computationally intractable when exposed to the combinatorial explosion of variables inherent to macroscopic strategy. A GOAP architecture, for example, functions by plotting a sequence of actions to reach a defined end-state. However, in a GSG where the action space to "improve the national economy" could theoretically involve interacting with five hundred distinct geographic provinces, fifty building types, twenty global trade nodes, and a hundred independent diplomatic relations, the algorithmic branching factor $O(b^d)$ expands beyond the processing capabilities of modern silicon.¹ Similarly, Hierarchical Behavior Trees become impossibly labyrinthine when tasked with handling thousands of shifting economic modifiers, technological unlocks, and cultural traits, resulting in deeply nested logic that is difficult for designers to parse and expensive for the CPU to traverse.

To resolve this intractability, modern macroscopic engines have abandoned rigid logic trees in favor of dynamic, data-driven Utility AI scoring pipelines heavily augmented by "Personality" frameworks.⁹ This approach, famously detailed in post-mortem analyses of titles like Stellaris, relies on mathematical weighting rather than hardcoded C++ decision branches.⁹

The Personality framework acts as the behavioral backbone and primary optimization filter for computer-controlled empires. Rather than an AI actor objectively analyzing the entirety of the game board to deduce the mathematically optimal move—a process that would rapidly homogenize opponent behavior and obliterate the simulation's replayability—the AI is assigned a personality archetype derived from its procedural starting parameters, such as its societal ethics, civic traits, or historical background.¹¹ These personalities serve as both computational constraints and baseline weight modifiers. An AI categorized under a "Hegemonic Imperialist" archetype will possess programmatic directives that heavily weight military expansion algorithms while entirely bypassing the computational evaluation branches associated with pacifist diplomatic federations.¹¹ By excising entire branches of the decision space before the utility pipeline even begins its calculations, the Personality framework simultaneously reduces CPU overhead and guarantees a thematic, human-like behavioral consistency.¹¹

The granular execution of this data-driven design treats every potential action an AI can take—constructing a localized planetary building, establishing an autonomous global trade route, or initiating a diplomatic defensive pact—as an independent object encapsulated within a relational database.¹¹ Each object is assigned a baseline mathematical weight. The AI evaluates these objects using an interconnected pipeline of "triggers," which are lightweight Boolean

checks evaluated against the current global game state.¹⁰ These triggers assess conditions such as verifying if an empire possesses a specific technology, if its population meets a certain ideological threshold, or if its resource stockpiles are secure.

When a decision interrupt fires, the engine evaluates the utility of all objects that successfully pass the initial Boolean gatekeeping. The final utility score for any given action is calculated by taking the baseline weight and multiplying it by a cascading series of conditional factors triggered by the current game state, effectively creating a massive, dynamic polynomial equation for every potential action.¹¹ This utility calculation can be expressed mathematically as follows:

$$U(a) = W_{base}(a) \prod_{i=1}^n F_i(C_i)$$

In this equation, $U(a)$ represents the final utility score of action a , $W_{base}(a)$ is the static baseline weight defined in the object database, and $F_i(C_i)$ represents the floating-point multiplier of a conditional trigger C_i if its corresponding Boolean check evaluates to true. For instance, if an AI is evaluating the construction of a military academy, the base weight might be set to 10. If the AI is currently engaged in an offensive war, a trigger identifies this state and applies a factor of 5.0, raising the utility to 50. Conversely, fractional overrides are strictly enforced to prune the decision tree dynamically; if a building is rendered obsolete by a new technological era or explicitly restricted by a newly enacted socio-political law, a conditional "allow" trigger will apply a factor of 0.0, immediately collapsing the utility score to zero and removing the object from the active evaluation pool.¹¹

Once all valid options are mathematically scored, the AI does not inherently default to the option with the highest $U(a)$. While strict Highest Weight Selection is reserved for critical, binary decisions that dictate the macro-strategic direction of a country—such as selecting national policies or designating an overarching geopolitical strategy—localized actions utilize a Random Weighted Algorithm.¹¹ This methodology normalizes the generated utility scores into a probability distribution, ensuring the AI remains fundamentally unpredictable and organic. The probability P of selecting action A from a pool of k valid actions is determined by the formula:

$$P(A) = \frac{U(A)}{\sum_{j=1}^k U(A_j)}$$

By relying on a probability distribution rather than deterministic maximization, the AI will occasionally select suboptimal but still heavily weighted choices. This mathematical stochastics elegantly simulates human quirkiness, flawed leadership, and organic geopolitical development

without requiring developers to engineer distinct, fragile "error-generation" algorithms.¹¹

The complexity of these economic and geopolitical utility pipelines is exponentially compounded when the AI is required to interface with autonomous macroeconomic systems, such as the global trade networks and private investment pools seen in advanced simulators like Victoria 3.¹³ In these environments, goods are not simply added to a unified national stockpile; rather, autonomous trade takes place on a World Market where every good possesses a dynamically fluctuating price based on the ratio of global export volumes versus import volumes.¹³ When an AI evaluates the utility of establishing a trade route, its scoring pipeline must calculate localized price derivatives, projecting the prospective market price alteration that its own hypothetical trade volume would inflict upon the network, and determining if the resultant profit margins exceed the convoy maintenance costs.¹³

It is worth noting the theoretical application of generative Large Language Models (LLMs) or Retrieval-Augmented Generation (RAG) frameworks—such as PlanRAG—to these macroscopic decision spaces. While recent academic endeavors have attempted to use LLMs as decision-makers in environments like Europa Universalis IV and Victoria 3, the latency, stochastic hallucination rates, and fundamental inability of LLMs to parse deeply interconnected numerical arrays make them entirely unsuitable for real-time evaluation.¹⁴ As engineers have noted, LLMs are fundamentally stochastic processes, and replacing robust, deterministic math with opaque, computationally ruinous generative inferences offers no functional advantage in a tightly bound state machine.¹⁸ Therefore, deterministic, programmatic utility equations remain the absolute gold standard for evaluating macroeconomic and geopolitical frontiers.

Integration with Concurrent Relational Registries

The sheer volume of mathematical utility evaluations required by tens of thousands of active AI agents necessitates heavy computational parallelization. However, macroscopic strategy games operate on a deterministic lockstep protocol, an uncompromising network architecture originally designed in the era of dial-up internet to synchronize peer-to-peer real-time strategy simulations—like Age of Empires and StarCraft—without transmitting massive, bandwidth-heavy game states.⁶ Under the deterministic lockstep paradigm, the central server does not dictate the state of the world to the clients. Instead, each client machine simulates the entire game locally, and only player inputs (or "commitments") are transmitted across the network for the current frame.⁶

To prevent look-ahead cheating—where a malicious client delays their own actions to await the messages of other players before reacting—the lockstep protocol requires each participant to broadcast a cryptographic hash value of their intended action before the action is executed.²¹ Because the underlying state machine is perfectly deterministic, utilizing fixed-point mathematics to avoid floating-point desynchronizations across different CPU architectures, all clients running the same inputs will arrive at the exact same systemic outcome frame by frame.⁶

While this architecture solves bandwidth limitations, it introduces a severe constraint on AI

computation: AI evaluations cannot be offloaded to asynchronous, non-deterministic threads. If one CPU thread evaluates an AI's economic choices slightly faster than another, and immediately mutates the global state before the second thread has finished reading it, a race condition occurs. This results in the local game state diverging from the rest of the network, causing an immediate out-of-sync (OOS) crash across the multiplayer matrix.²³

Early multi-threading attempts within grand strategy architectures, particularly those utilized during the initial iterations of the Clausewitz engine for titles like Crusader Kings II and Hearts of Iron IV, attempted to navigate this constraint by utilizing Threading Building Blocks (TBB) libraries.⁸ This methodology focused on parallelizing the world simulation by chunking entities. The engine would deploy `parallel_for()` loops across distinct sub-systems, such as updating all countries, then all military units, then all provinces.⁸ However, this "entity grain size" approach resulted in notoriously poor CPU scaling in late-game scenarios. Because the simulation was bound by a sequence, a system update could only advance as fast as its slowest individual entity. A massive, late-game empire encompassing half the globe would stall its respective thread with thousands of AI unit calculations, forcing all other available cores to sit idle at a synchronization barrier, waiting for the slowest thread to finish before the global state could advance.⁸

To achieve deep CPU saturation without violating the strict mandates of deterministic logic, modern architectures—such as the Jomini engine powering Crusader Kings III and Victoria 3—have implemented a rigorous two-phase "Read-Lock / Write-Lock" concurrency model.⁸ This paradigm fundamentally reorganizes how AI interacts with the global relational registry.

The architectural timeline of a single game tick is strictly bifurcated. During the initial **Read-Lock Phase**, the global game state is fundamentally frozen. The engine spawns a massive array of parallel tasks across the main session thread and all available worker cores.⁸ During this phase, AI systems execute the heaviest computational lifting: calculating utility scores, executing non-Euclidean pathfinding algorithms, evaluating triggers, and determining their intended strategic actions. Crucially, during the Read-Lock phase, AI systems are granted strictly read-only access to the global relational registry.⁸ An AI can query the treasury of a neighboring nation, assess border friction values, calculate global market price derivatives, and read current diplomatic statuses, but it is entirely forbidden from modifying those global variables. Any data generated during this phase—such as an AI's definitive decision to declare war, construct a building, or transfer resources—is written exclusively to transient, thread-local input buffers.⁸ Because no thread is allowed to mutate shared data, the CPU can execute these complex AI evaluations simultaneously across all 16 or 32 cores of a modern processor without any risk of mutex contention or data races.

Following a strict synchronization barrier where all threads confirm completion of their read operations, the engine transitions into the **Write-Lock Phase**.⁸ During this phase, parallelization is largely collapsed. The engine sequentially reads the transient input buffers generated by the AI actors (alongside any networked human player commands) during the preceding phase and applies their mutations to the global state in a strictly deterministic, predefined execution order.⁸

While this phase relies on single-threaded, sequential execution to guarantee absolute determinism, it is computationally lightweight. The CPU is merely flipping flags, transferring integer values between arrays, and initiating state transitions; the computationally ruinous tasks—the deep pathfinding, the cascading polynomial utility calculations, the spatial queries—have already been resolved concurrently.

Architectural Phase	Mutex and State Status	Threading Paradigm	Primary Computational Operations
Read-Lock Phase	Global State Read-Only	Highly Parallel (All Worker Threads)	Utility scoring pipelines, pathfinding, trigger evaluation, local buffer generation.
Synchronization Barrier	Await All Threads	Engine Synchronization	Ensures all transient AI commands and player hashes are finalized and collected.
Write-Lock Phase	Global State Writable	Sequential (Main / Session Thread)	State mutation, deterministic event firing, command execution, integer transfers.

While the CK3 model is highly effective, averaging utilization of roughly 5 out of 16 cores (a massive improvement over legacy single-core bottlenecks), Paradox Development Studio engineers have outlined the trajectory toward even more granular, task-based Data-Driven Multi-Threading Schedulers for future engine iterations.⁸ By forcing every AI script to explicitly declare its read and write dependencies at compile time using thin accessors—lightweight pointers that define data requirements before deep object dereferencing occurs—the engine can construct a dynamic Directed Acyclic Graph (DAG) of concurrent tasks.⁸ If the scheduler determines that two AI logic tasks operate on entirely disparate memory domains, it can safely schedule them to run concurrently even during the mutation phase, further shattering the serialization bottleneck.

State-Polling and Event-Driven Sensibility

In a simulation featuring thousands of dynamic variables—ranging from localized planetary unrest and shifting demographic demographics to global market inflation and cascading war exhaustion—an AI cannot actively query the entire game state continuously. If every AI agent polled a matrix of a million relational properties at 60 ticks per second to determine its next action, it would instantly bottleneck the memory bus, flood the L1/L2 caches, and crash the simulation. Therefore, macroscopic AI systems invert the relationship between the computational observer and the data, relying on event-driven interrupts, arithmetic triggers, Mean Time To Happen (MTTH) distributions, and distributed state-polling.

Rather than the AI actively asking, "Has my economic deficit reached critical levels?" on every single frame, the underlying mathematical framework of the engine utilizes specific arithmetic triggers that evaluate conditions directly at the point of state mutation.¹⁰ Engine features such as `check_variable_arithmetic` allow the game to perform mathematical evaluations seamlessly in the background.¹⁰ When a player or AI action causes a variable—such as an energy resource stockpile—to drop below zero, the stockpile management system detects this delta threshold crossing and fires an interrupt directly to the owning AI's controller. This interrupt breaks the AI out of its idle optimization loop, instantly injecting a high-priority "deficit management" node into its utility pipeline.²⁷ This ensures the AI reacts immediately to critical existential threats without wasting CPU cycles verifying its safety during the thousands of ticks where its economy is stable.

For less critical, long-term strategic evaluations, the AI relies on the Mean Time To Happen (MTTH) architecture.²⁸ MTTH is a sophisticated probabilistic polling matrix that evaluates events on a delayed, staggered schedule rather than a continuous loop.²⁸ If an AI empire occupies a hostile territory, the decision to install a collaborative government, increase military policing, or initiate a purge is governed by an MTTH block. The baseline MTTH for an evaluation might be set to 30 days. Rather than running a full evaluation every day, the engine calculates the daily probability of this logic block triggering based on the formula:

$$P(trigger) = 1 - 2^{-\frac{t}{MTTH}}$$

Where t is the time elapsed. This baseline probability is dynamically altered by background game state modifiers. For instance, if the resistance level in the occupied zone spikes due to foreign espionage, a modifier might divide the effective MTTH by half, vastly increasing the mathematical likelihood of the AI intervening.²⁸ This probabilistic polling ensures that the AI reacts organically to shifting geopolitical realities, distributing the computational load of deep state evaluations unpredictably across the timeline rather than executing them simultaneously.

Furthermore, macroscopic games frequently employ "storylet" architectures to bridge the gap between shifting mathematical variables and discrete narrative or diplomatic actions.³⁰ Storylets are encapsulated chunks of pre-authored behavioral content with deep, complex preconditions

attached.³⁰ The AI utilizes these storylets to interface with human players and other AI actors in a way that feels intentional and contextually aware. Instead of the AI continuously polling all neighbors to see if it should send an insult or propose a royal marriage, the engine maintains a passive registry of active storylets. When the background state aligns with the preconditions of a storylet—for example, if border tension exceeds a specific arithmetic threshold, religious hostility is high, and a valid *casus belli* exists—the system surfaces the interaction directly to the AI's highest-weight selection pipeline.³⁰ This event-driven sensibility guarantees that complex AI behaviors occur precisely when the mathematical tension of the simulation demands it, creating the illusion of deep strategic foresight and narrative continuity while requiring a fraction of the processing power of continuous search algorithms.

Topological Pathfinding and Strategic Maneuvering

Strategic maneuvering in a GSG differs radically from the pathfinding paradigms utilized in standard RTS or action games. The spatial graphs in titles like *Hearts of Iron IV*, *Crusader Kings III*, or *Europa Universalis* are strictly non-Euclidean.⁵ They represent interconnected political borders, abstract naval seasons, and topological supply networks that do not map sequentially with flat spatial geometry.³³ Moving a digital army from London to Berlin is not a simple calculation of two-dimensional Euclidean distance, but rather a complex traversal of a connected mathematical graph where the "distance" (the edge weight between nodes) is dictated by abstract concepts such as naval supremacy, infrastructure degradation, shifting weather patterns, and localized attrition.³³

To solve these complex pathfinding scenarios across a mutating global map, the AI architecture utilizes highly specialized algorithms focused on front-line management and dynamic edge weighting. In environments simulating total war, such as *Hearts of Iron IV*, AI actors do not attempt to micromanage individual divisions as independent, autonomous agents. Instead, the AI operates at the operational command level, utilizing a "Battle Plan" abstraction.³⁵ The AI defines a geometric spline across the topological graph—representing a continuous front line against a hostile nation—and assigns an aggregate mass of unit objects to this abstraction.³⁵

Once the front line is drawn, the AI calculates the optimal distribution of its forces along this spline using a density-weighting algorithm.³⁷ The algorithm evaluates every province node along the spline and assigns dynamic importance weights based on tactical vulnerability and strategic value. A province node containing a major supply hub, an airfield, or one situated in flat terrain vulnerable to a mechanized armored breakthrough, is given an exceptionally high weight density.³⁶ Conversely, mountainous nodes that are easily defensible require a lower density of troops. The AI then utilizes a heavily modified A* (A-Star) search algorithm to route individual divisions from the rear echelons to fulfill the required weight density of the front line.³⁴

Crucially, the heuristic utilized by this A* algorithm must account for extremely volatile edge weights. If an AI division begins marching toward a high-density node, but an enemy bomber wing destroys the railway infrastructure in an intermediate province, the traversal cost of that specific edge spikes dramatically. The AI pathfinding algorithm must continually recalculate the

graph to avoid "supply-induced suicide"—a scenario where hundreds of thousands of digital soldiers exhaust their organization and equipment by routing through an over-encumbered or destroyed supply node.³⁷

To optimize this continuous recalculation without halting the CPU, spatial partitioning and hierarchical pathfinding frameworks are aggressively implemented.⁴⁰ The global map is subdivided into macro-regions or logical sectors.⁴⁰ When an AI needs to route a reinforcement division from the shores of France to the steppes of Russia, it does not calculate the cost of every granular province in between. Instead, the algorithm first operates on a simplified macro-graph, finding a generalized route through connecting sectors (e.g., Western Europe -> Central Europe -> Eastern Europe).⁴⁰ Only when the unit actively enters a specific sector does the AI expend the processing power to calculate the precise, province-to-province micro-pathing required to navigate local terrain and combat zones.

Furthermore, the integration of naval and land transport combinations introduces massive complexity to the pathfinding matrix.³⁴ The AI must possess the computational capacity to evaluate if a land route suffering from high passive periodic damage (attrition) is mathematically superior to embarking upon a naval invasion across a sea-zone shadowed by a hostile naval presence.³⁴ This requires cross-domain graph traversal, where the AI must calculate the utility penalty of embarking and disembarking, the statistical probability of convoy interception, and weigh it against the temporal and organizational costs of terrestrial movement.

The introduction of granular tactical mechanics, such as "combat width"—which strictly limits the number of units that can actively engage in combat within a single province node—forces the AI to abandon the traditional strategy of consolidating forces into a single, overwhelming "Stack of Doom".³⁶ Instead, the AI's front-line management algorithm must continuously distribute and cycle units across the topology, optimizing for maximum combat width saturation while explicitly avoiding exceeding local supply limits or stacking penalties.³⁶ This requires the AI to maintain a constant, delicate equilibrium of mathematical weights across a non-Euclidean front that shifts violently with every discrete tick of the simulation.

CPU Saturation, Modulo-Based Scheduling, and Load Balancing

The ultimate bottleneck in grand strategy AI development is raw CPU time. While the Read-Lock concurrency model successfully parallelizes the workload across multiple physical cores, evaluating the complex utility equations, macroeconomic trade routes, and non-Euclidean pathfinding vectors for 10,000 distinct AI actors within a strict 16.6-millisecond frame window (the requirement for a 60 FPS simulation) is mathematically impossible on consumer hardware. If the simulation attempts to update every actor simultaneously, the frame time will spike drastically, resulting in a phenomenon known as "stuttering," where the temporal serialization loop is visibly stalled.

Therefore, the underlying engine architecture relies heavily on rigorous time-slicing and

modulo-based scheduling algorithms to distribute the cognitive load evenly over the temporal axis.¹⁰ Modulo scheduling is an elegant load-balancing technique that ensures the serialization loop is never stalled by a massive, synchronized spike in concurrent AI evaluations. Instead of instructing all AI actors to evaluate their economic utility pipelines on the first tick of the month or the first day of the year, the engine utilizes the mathematical modulo operator against the AI actor's unique internal ID integer and the current global tick count.¹⁰

The core logic of this scheduling can be conceptualized as:

```
if (current_tick % evaluation_interval == actor_id % evaluation_interval)
```

If the `evaluation_interval` is set to 30 ticks, the engine essentially creates 30 distinct temporal buckets. An AI nation or demographic population (a "pop") with an ID of 104 will evaluate its economy on tick 14, tick 44, tick 74, and so forth, because $104 \pmod{30} = 14$. An actor with an ID of 105 will evaluate on tick 15. By uniformly distributing the AI evaluations across the defined interval, the CPU handles a steady, highly predictable stream of calculations rather than violent, frame-halting spikes, ensuring deep optimization of the instruction cache.¹⁰

This modulo scheduling logic is further optimized by dynamically scaling the `evaluation_interval` based on Level of Detail (LOD) principles applied to computational frequency rather than visual rendering.⁴⁰ Not all AI actors require the same frequency of thought. An AI empire currently engaged in a massive two-front war against a human player requires high-frequency tactical evaluation; its front-line management script might be scheduled with a very tight interval, running every 5 ticks to ensure it reacts promptly to player breakthroughs.⁴⁰ Conversely, a minor, isolated AI nation on the opposite side of the globe operating in a time of peace has its `evaluation_interval` drastically expanded, perhaps evaluating its strategic goals and economic buildings only once every 100 or 200 ticks.⁴⁰

When the modulo trigger successfully fires for an actor, the AI's evaluation payload is heavily chunked. Rather than executing a monolithic block of C++ code that locks the thread, the AI queues specific cognitive tasks—such as `Evaluate_Diplomacy`, `Pathfind_Fleet`, or `Rebalance_Trade`—into the engine's overarching multi-threading scheduler.⁸ The data-driven scheduler processes these tasks utilizing the aforementioned thin accessors, which act as lightweight memory pointers that intentionally bypass deep object dereferencing until the specific data is absolutely required by the CPU.⁸ By executing evaluations in staggered, time-sliced, parallelized chunks, the engine effectively guarantees that the game state advances seamlessly. This complex choreography hides the sheer volume of mathematical computations occurring beneath the surface, enabling the simulation of macroscopic geopolitical history without suffocating the hardware that executes it.

Works cited

1. MCTS-based Planning for Grand Strategy Games, accessed April 16, 2026, <https://fenix.tecnico.ulisboa.pt/downloadFile/1126295043839104/94042-Manuel-R>

- [oxo_resumo.pdf](#)
2. (PDF) AI for General Strategy Game Playing - ResearchGate, accessed April 16, 2026,
https://www.researchgate.net/publication/275963230_AI_for_General_Strategy_Game_Playing
 3. A guided tour of artificial intelligence research. Vol 2 9783030061661, 9783030061678 - DOKUMEN.PUB, accessed April 16, 2026,
<https://dokumen.pub/a-guided-tour-of-artificial-intelligence-research-vol-2-9783030061661-9783030061678.html>
 4. A Review of Real-Time Strategy Game AI - AAAI Publications, accessed April 16, 2026,
<https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/download/2478/2457>
 5. 2017 - Eye of Hydra, accessed April 16, 2026,
<http://zenorogue.blogspot.com/2017/>
 6. pietrobassi/deterministic-lockstep-demo - GitHub, accessed April 16, 2026,
<https://github.com/pietrobassi/deterministic-lockstep-demo>
 7. Deterministic Lockstep | Gaffer On Games, accessed April 16, 2026,
https://gafferongames.com/post/deterministic_lockstep/
 8. Multi Threading Model in Paradox Games - ACCU 2023, accessed April 16, 2026,
https://accu.org/conf-docs/PDFs_2023/XMultiThreadingModelinParadoxGamesPastPresentandFuture.pdf
 9. Video: Creating Stellaris' complex AI behavior via data-driven design, accessed April 16, 2026,
<https://www.gamedeveloper.com/design/video-creating-i-stellaris-i-complex-ai-behavior-via-data-driven-design>
 10. accessed April 16, 2026,
https://store.steampowered.com/news/posts/?feed=steam_community_announcements&appids=281990&enddate=1616075182
 11. Creating Complex AI Behavior in Stellaris Through Data-Driven Design - YouTube, accessed April 16, 2026, <https://www.youtube.com/watch?v=Z5LMUbjyFQM>
 12. Creating Complex AI Behavior in 'Stellaris' Through Data Driven Design - GDC Vault, accessed April 16, 2026,
<https://www.gdcvault.com/play/1024223/Creating-Complex-AI-Behavior-in>
 13. Victoria 3 - Dev Diary #143 - Trade Rework: The World Market : r/victoria3 - Reddit, accessed April 16, 2026,
https://www.reddit.com/r/victoria3/comments/1jl5yhi/victoria_3_dev_diary_143_trade_rework_the_world/
 14. Decision-Oriented Dialogue for Human-AI Collaboration - ResearchGate, accessed April 16, 2026,
https://www.researchgate.net/publication/382823093_Decision-Oriented_Dialogue_for_Human-AI_Collaboration
 15. Daily Papers - Hugging Face, accessed April 16, 2026,
<https://huggingface.co/papers?q=decision%20frontiers>
 16. On the Fundamental Limits of LLMs at Scale - arXiv, accessed April 16, 2026,
<https://arxiv.org/html/2511.12869v1>

17. awesome-generative-ai-guide/research_updates/rag_research_table.md at main - GitHub, accessed April 16, 2026,
https://github.com/aishwaryanr/awesome-generative-ai-guide/blob/main/research_updates/rag_research_table.md
18. We'll never have a model of an AI major-general: Artificial Intelligence, command decisions, and kitsch visions of war - PMC, accessed April 16, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10941604/>
19. Anthropic engineer says "software engineering is done" first half of next year - Reddit, accessed April 16, 2026,
https://www.reddit.com/r/ClaudeAI/comments/1p771rb/anthropic_engineer_says_software_engineering_is/
20. Techniques and Paradigms in Modern Game AI Systems - MDPI, accessed April 16, 2026, <https://www.mdpi.com/1999-4893/15/8/282>
21. Lockstep protocol - Wikipedia, accessed April 16, 2026,
https://en.wikipedia.org/wiki/Lockstep_protocol
22. Deterministic Lockstep Networking Demystified - Zack Sinisi – Game Programmer | C++, C# | UE4, UE5, Unity, accessed April 16, 2026,
<https://zacksinisi.com/deterministic-lockstep-networking-demystified/>
23. accessed April 16, 2026,
https://store.steampowered.com/news/posts/?feed=steam_community_announcements&appids=281990&enddate=1603991604
24. What are some common misconceptions you see on Reddit about gamedev?, accessed April 16, 2026,
https://www.reddit.com/r/gamedev/comments/159d4wd/what_are_some_common_misconceptions_you_see_on/
25. Crusader Kings 3 multicore optimization is perfect, please implement it in your other games! : r/paradoxplaza - Reddit, accessed April 16, 2026,
https://www.reddit.com/r/paradoxplaza/comments/jgq363/crusader_kings_3_multicore_optimization_is/
26. accessed April 16, 2026,
https://store.steampowered.com/news/posts/?feed=steam_community_announcements&appids=281990&enddate=1619092780
27. Collections: Teaching Paradox, Crusader Kings III, Part I: Making It Personal, accessed April 16, 2026,
<https://acoup.blog/2022/09/16/collections-teaching-paradox-crusader-kings-iii-part-i-making-it-personal/>
28. AI modding - Hearts of Iron 4 Wiki, accessed April 16, 2026,
https://hoi4.paradoxwikis.com/AI_modding
29. Patch 1.9.1 "Husky" Update Summary | PDF | Spanish Civil War | Portugal - Scribd, accessed April 16, 2026,
<https://www.scribd.com/document/501465099/changelog>
30. Notes from the Boundaries of Interactive Storytelling - Polaris Game Design Retreat, accessed April 16, 2026,
<https://polarisgamedesign.com/2024/notes-from-the-boundaries-of-interactive-storytelling/>

31. FAQ - CK3 Wiki, accessed April 16, 2026, <https://ck3.paradoxwikis.com/FAQ>
32. Steam 创意工坊::2024 HOTEL, accessed April 16, 2026, <https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3235912465>
33. Architectonics of Game Spaces - The Spatial Logic of the Virtual and Its Meaning for the Real - Oapen, accessed April 16, 2026, https://library.oapen.org/bitstream/id/5485c636-ab78-40c3-a1c7-6bf5a94a2bf9/external_content.pdf
34. Patch 1.5.X - Hearts of Iron 4 Wiki, accessed April 16, 2026, https://hoi4.paradoxwikis.com/Patch_1.5.X
35. Battle plan - Hearts of Iron 4 Wiki, accessed April 16, 2026, https://hoi4.paradoxwikis.com/Battle_plan
36. Complete Army Guide for Beginners in Hearts of Iron IV! - YouTube, accessed April 16, 2026, <https://www.youtube.com/watch?v=a7xz8RaOotw>
37. Steam Workshop::Better Mechanics : Frontline AI | Updated, accessed April 16, 2026, <https://steamcommunity.com/sharedfiles/filedetails/?id=2984937408>
38. What can we learn about AGI from game AI? - Julian Togelius, accessed April 16, 2026, <http://julian.togelius.com/Togelius2026What.pdf>
39. How is the AI and realism (been hearing bad things) :: Hearts of Iron IV General Discussions, accessed April 16, 2026, <https://steamcommunity.com/app/394360/discussions/0/14451287532398805/>
40. Advice for building AI system that can handle a large number of NPCs/entities? - Reddit, accessed April 16, 2026, https://www.reddit.com/r/roguelikedev/comments/1di80wc/advice_for_building_ai_system_that_can_handle_a/
41. Steam Workshop::自用(大杂烩) (偏科技), accessed April 16, 2026, <https://steamcommunity.com/sharedfiles/filedetails/?id=3126315783>
42. Concept Preview: Combat System Overhaul (Part 2 to Part) [UNCLARIFIED], accessed April 16, 2026, <https://forums.civfanatics.com/threads/concept-preview-combat-system-overhaul-part-2-to-part-unclarified.670706/>
43. 2005 CAP CONGRESS / CONGRÈS DE L'ACP 2005 June 5-8 juin UBC, Vancouver, BC, accessed April 16, 2026, <https://www.cap.ca/wp-content/uploads/2017/03/a47a3c05b05716188c998d919757f57293e56643-1.pdf>
44. A Study of the Production Planning and Scheduling Problem in the Apparel Manufacturing Industry. Phase 2 - DTIC, accessed April 16, 2026, <https://apps.dtic.mil/sti/tr/pdf/ADA286836.pdf>